

Summary

Allixir Intelligence is an AI-powered mobile application that transforms unstructured documents into structured, actionable knowledge. Users can upload or scan documents (e.g., medical reports, invoices), from which the system extracts domain-specific, meaningful data fields and organises them into a personalised knowledge base. A voice/chat interface enables users to ask precise, context-aware questions. Responses are generated using retrieval-augmented generation (RAG) over the user's personal knowledge base, with optional enrichment from external knowledge sources such as research papers.

Technology Stack

Frontend

- React Native: Framework for native apps using React and TypeScript
- Expo: a React Native platform that provides ready-to-use APIs such as camera functionality
- Tamagui: cross-plattform UI kit and style compiler that optimizes performance by flattening component trees
- TanStack Query: data fetching with caching, invalidation and request deduping
- Jotai: Atom-based state management library

Database Tier

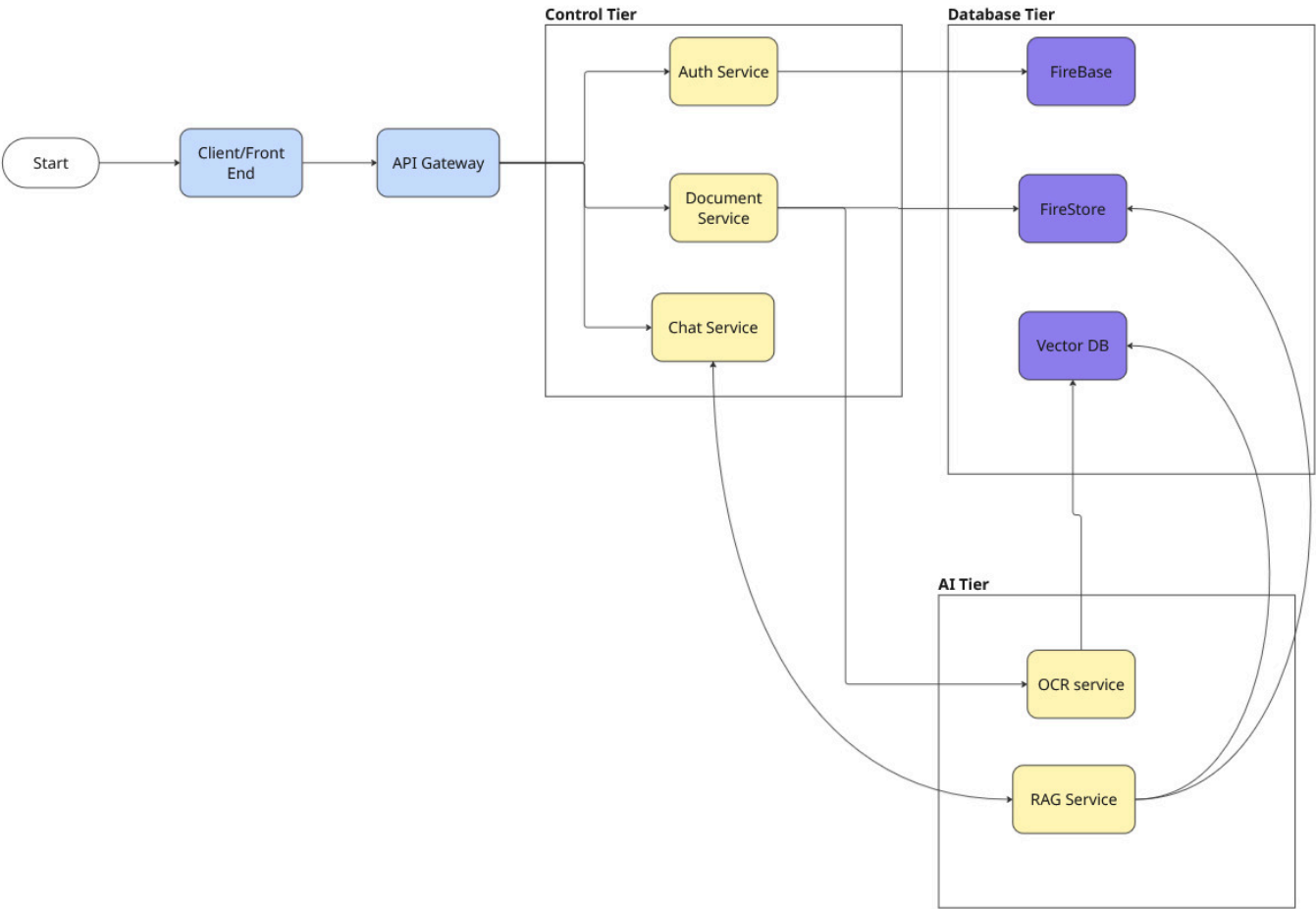
- Cloud Firestore: User-specific data storage of the frontend app (e.g. chat history, ...)
- Vector DB: Storage of backend data (e.g. from RAG, extracted data from OCR, knowledge graphs, ...).

AI Tier

- OCR models: Data extraction from documents with Google Document AI or open source options such as PaddleOCR and QwenOCR
- RAG Service: LangChain/LangGraph for context-aware conversation handling
- Knowledge Graphs: Implemented using LangGraph. In case of complex relations we may use Neo4j or Graphiti

Control Tier

- Firebase: User authentication for the frontend app, as per recommendations of the industry partner
- Google Cloud Run: Backend services can be run and will be exposed using an API gateway



Frontend Architecture

Tech stack

App runtime

Technology	Purpose
Expo SDK 55	Mobile and web application runtime
React 19	UI rendering model
React Native 0.83	Native view layer

State and data

Technology	Purpose
Jotai	Auth state and lightweight global state
TanStack Query	Server-state cache, refetching, and request lifecycle
Axios	HTTP client for backend API calls

UI system

Technology	Purpose
Tamagui	Cross-platform styling and layout primitives

Backend integration

Technology	Purpose
Firebase Auth	User identity and session state
Firebase client SDK	Auth listener and sign-in/out flows

Frontend Repository layout

```
src/
├── app/                    # expo-router screens and layouts
│   ├── (auth)/            # login and signup flow
│   └── (private)/         # authenticated app experience
├── components/            # atoms, molecules, organisms
├── hooks/                 # data and upload hooks
├── lib/                   # Firebase, axios, and auth state
├── tamagui/              # tokens, themes, and UI configuration
└── constants/            # copy and typography helpers
```

Route protection

Route protection is handled with `Stack.Protected` guards:

- authenticated users go to the `(private)` stack
- unauthenticated users go to the `(auth)` stack

This keeps the auth boundary in one place instead of scattering it across screens.

Public/auth screens

The auth stack under `src/app/(auth)/` contains the sign-in and sign-up flow, which can be accessed before the user is authenticated

Private screens

The private stack under `src/app/(private)/` contains the main application experience:

- `index.tsx` is the dashboard with quick actions and recent chats
- `documents/index.tsx` is the document overview and entry point for upload/capture flows
- `documents/[id].tsx` shows a single document
- `documents/upload.tsx` and `documents/capture.tsx` handle file acquisition
- `chats/index.tsx` and `chats/[id].tsx` handle chat navigation
- `settings.tsx` shows the signed-in user and logout action

Component architecture

The codebase uses a small design system pattern:

- atoms in `src/components/atoms/`
- molecules in `src/components/molecules/`
- organisms in `src/components/organisms/`

Backend Architecture

The Ailixir backend is split into two Cloud Run services: a **synchronous API** the mobile app talks to, and an **asynchronous worker** that runs the AI extraction pipeline. They communicate through Google Cloud Pub/Sub. Persistence is in Firestore; binary content lives in Google Cloud Storage. Extracted knowledge is stored as a Neo4j knowledge graph populated by Graphiti with Vertex AI Gemini for entity extraction.

Tech stack

API service

Technology	Description
Python 3.11	Language runtime
FastAPI 0.136	Async HTTP framework with auto-generated OpenAPI / Swagger UI
Pydantic v2	Request/response validation, model schema enforcement
Firebase Admin SDK 7.4	Verifies Firebase ID tokens, creates user accounts
google-cloud-storage 3.10	Generates v4 signed upload/download URLs
google-cloud-pubsub 2.27	Publishes <code>DocumentUploaded</code> events
Uvicorn 0.46	ASGI server inside the container

Worker service

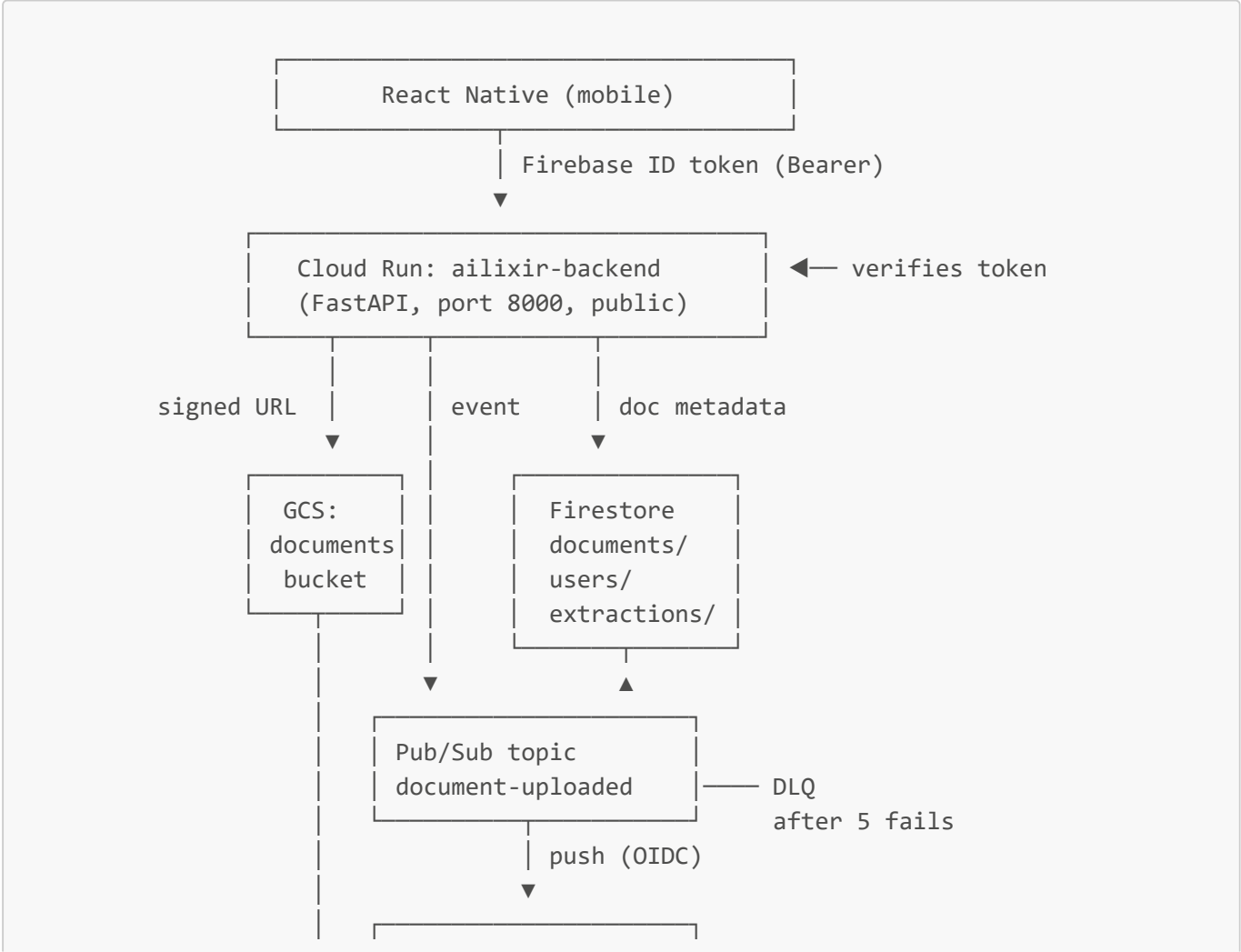
Technology	Description
Python 3.11	Language runtime
FastAPI 0.136	Receives Pub/Sub push HTTP requests
Google Cloud Document AI	OCR for PDFs and images (replaces hand-rolled vision-LLM OCR)
Vertex AI Gemini	LLM-based entity/relationship extraction inside Graphiti (<code>gemini-2.5-flash-lite</code>) and embeddings (<code>text-embedding-005</code>)
Graphiti 0.3+	Knowledge-graph construction over Neo4j
Neo4j 5+	Graph database for extracted entities and relationships

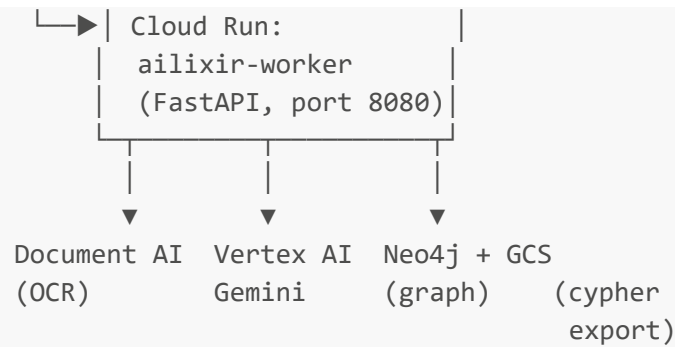
Technology	Description
google-cloud-storage 2.19	Downloads uploaded documents, uploads exported Cypher files

Infrastructure

Technology	Description
Google Cloud Run	Hosts both services (<code>ailixir-backend</code> , <code>ailixir-worker</code>)
Google Cloud Storage	Two buckets — uploaded documents, exported Cypher graphs
Google Cloud Pub/Sub	<code>document-uploaded</code> topic + DLQ + push subscription
Cloud Firestore	NoSQL document store (<code>documents</code> , <code>users</code> , <code>extractions</code> collections)
Firebase Auth	Identity and password handling
Terraform 1.5	Infrastructure as code, split state per service
GitHub Actions	CI/CD — lint, build, push, terraform apply
Google Container Registry	Stores <code>ailixir-backend</code> and <code>ailixir-worker</code> images

Architecture





A companion diagram covering the same components at a packaging level lives at [Documentation/architecture/code_components_diagram.pdf](#).

Repository layout

```

Backend/
├── api/                                # FastAPI service, public HTTP entry
│   ├── auth.py                        # Firebase ID-token verification
│   ├── errors.py                      # Structured error envelope
│   ├── main.py                        # Routes + Pydantic request/response models
│   ├── middleware.py                 # X-Request-ID propagation
│   ├── requirements.txt
│   ├── Dockerfile
│   └── terraform/
├── workers/                           # API-only infrastructure
│   ├── connections/                 # Pub/Sub push receiver + AI pipeline
│   ├── pipeline/
│   │   ├── document_pipeline.py     # End-to-end orchestrator
│   │   ├── ocr/                     # Google Cloud Document AI
│   │   └── graph/                   # Graphiti episode builder + Cypher exporter
│   ├── main.py                      # /pubsub/push endpoint
│   ├── requirements.txt
│   ├── Dockerfile
│   └── terraform/
├── shared/                           # Worker-only infrastructure
│   ├── firestore.py                 # Used by both services
│   ├── gcs.py                       # Firestore client singleton
│   ├── pubsub.py                    # Signed URL helpers
│   ├── models/                      # Event publisher
│   └── repositories/                # Pydantic domain models
├── firestore.indexes.json            # Firestore data access (per collection)
└── .env.example                     # Composite indexes for the documents collection
# Template for local .env
  
```

Components

API service ([Backend/api/](#))

Synchronous HTTP endpoint for the mobile client. Tuned for low latency — bytes never stream through the service.

- **Routes** (`api/main.py`)
 - `POST /auth/signup` — atomic Firebase Auth + Firestore profile create.
 - `GET /me` — current user profile.
 - `POST /documents` — create document metadata, return one signed PUT URL per file.
 - `POST /documents/{id}/upload-urls/refresh` — reissue URLs for files that haven't been uploaded yet.
 - `POST /documents/{id}/finalize` — verify GCS objects exist, transition to `uploaded`, publish event.
 - `GET /documents` — paginated, cursor-based list with filters (`domain`, `status`).
 - `GET /documents/{id}` — single document plus signed download URLs.
 - `DELETE /documents/{id}` — soft delete (sets `deleted_at`).
 - `GET /health` — liveness probe.
- **Authentication** (`api/auth.py`) — every authenticated route depends on `get_current_user`, which calls `firebase_admin.auth.verify_id_token` on the Bearer header.
- **Request ID middleware** (`api/middleware.py`) — pure ASGI middleware that attaches `X-Request-ID` to every response and propagates it through `contextvars` into every log line.
- **Error handling** (`api/errors.py`) — every error response uses a single JSON envelope `{ "error": { "code", "message", "request_id" } }` with stable machine-readable codes from `shared/models/errors.py`.

Worker service (`Backend/workers/`)

Pub/Sub push receiver. Runs the AI pipeline end-to-end inside a single request so Cloud Run keeps the instance alive for the full job.

- **HTTP entry** (`workers/main.py`) — `POST /pubsub/push` verifies the OIDC token, decodes the base64 Pub/Sub envelope, dispatches by `event_type`. Currently the only handler is `DocumentUploaded`.
- **Pipeline orchestrator** (`workers/pipeline/document_pipeline.py`) — loads the document from Firestore, then per file: download → OCR → save extraction → build graph episode → export Cypher → upload Cypher → mark `extracted`. Any exception transitions the document to `failed` with the error message preserved.
- **OCR adapter** (`workers/pipeline/ocr/document_ai.py`) — calls Google Cloud Document AI with magic-byte MIME-type detection so GCS metadata gaps don't break extraction.
- **Graph builder** (`workers/pipeline/graph/builder.py`) — wraps each document as a Graphiti episode; Graphiti's internal Gemini calls extract entities and relationships into Neo4j.
- **Cypher exporter** (`workers/pipeline/graph/exporter.py`) — walks the episode's subgraph two hops out, renders a self-contained `.cypher` script, uploads to the cypher bucket. The Firestore document gets a `cypher_gcs_uri` pointer the frontend reads.
- **Connection singletons** (`workers/connections/`) — single Neo4j driver and Graphiti client, lazily built on first request, kept alive across deliveries.

Shared layer (`Backend/shared/`)

- **Firestore client** (`shared/firestore.py`) — three-branch credential resolution: emulator, ADC (empty key path), or service-account JSON. Singleton, thread-safe.

- **GCS client and signed URL helpers** ([shared/gcs.py](#)) — generates v4 signed URLs bound to method, content-type, max size, and `if-generation-match: 0` (write-once). Uses IAM Credentials API on Cloud Run for signing without a private key.
- **Pub/Sub publisher** ([shared/pubsub.py](#)) — publishes `DocumentUploaded` events with `event_id`, `schema_version`, ordering key on `document_id`.
- **Domain models** ([shared/models/](#)) — `Document`, `DocumentFile`, `DocumentStatus` enum, `UserProfile`, `Extraction`, `ErrorCode` enum. Enforced limits: 50 files / document, 20 MB / file, 200 MB / document total.
- **Repositories** ([shared/repositories/](#)) — Firestore access for `documents`, `users`, `extractions`. Authorisation by `uid` lives here (the API can't bypass it). `mark_uploaded` is transactional with status precondition; `soft_delete` refuses if the document is currently in `processing`.

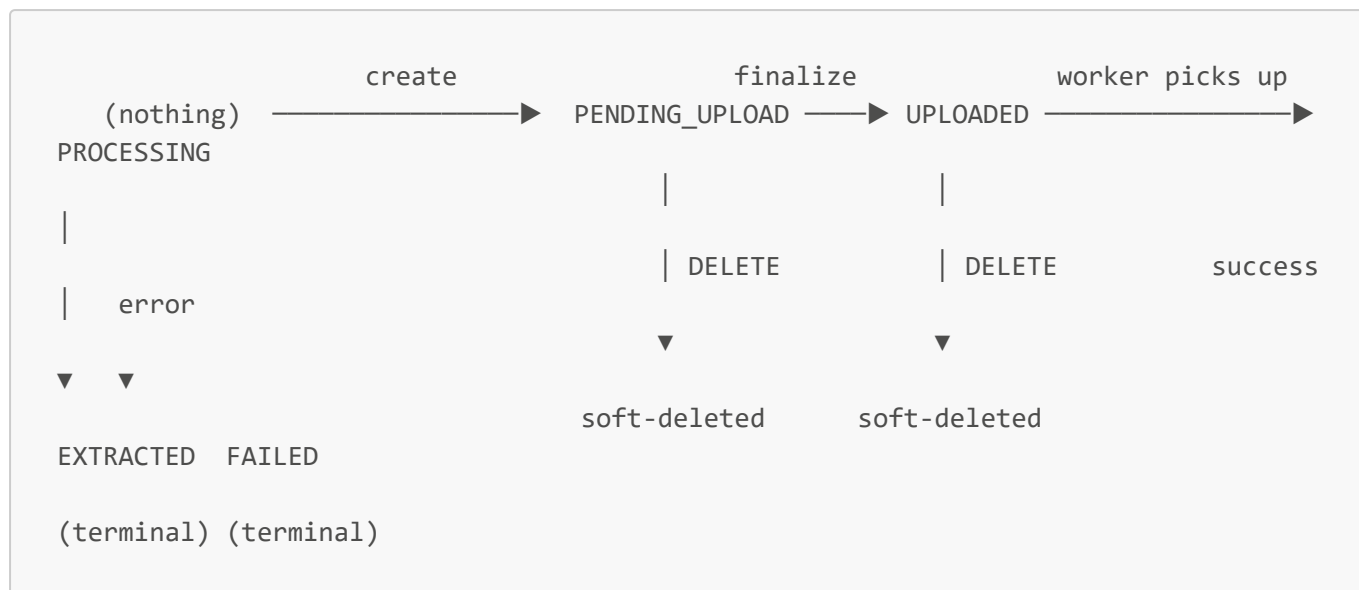
Data flow — uploading a document

1. Client → POST `/documents`
API creates Firestore document(s), generates one signed PUT URL per file.
Status: `pending_upload`
2. Client → PUT bytes directly to each signed URL (GCS)
Bytes never touch the API. Signature enforces content-type, max size, write-once.
3. Client → POST `/documents/{id}/finalize`
API HEAD-checks each GCS object, stamps `upload_completed_at` per file in a Firestore transaction, transitions status: `pending_upload` → `uploaded`.
Then publishes a `DocumentUploaded` event to Pub/Sub.
4. Pub/Sub → POST `/pubsub/push` on the worker (with OIDC Bearer)
Worker verifies token issuer + audience + service-account email + `email_verified`.
5. Worker pipeline (synchronous, blocks until done)
PROCESSING
 - ↓ for each uploaded file:
 - download from GCS → OCR via Document AI
 - ↓
 - Save Extraction document to Firestore (extractions collection)
 - ↓
 - Graphiti `add_episode` → Gemini extracts entities/relationships → Neo4j
 - ↓
 - Walk episode subgraph → render `.cypher` → upload to cypher bucket
 - ↓
 - Attach `cypher_gcs_uri` to the Firestore document
 EXTRACTED

Any exception → FAILED with error message recorded.

The status sequence is implemented by the `DocumentStatus` enum in [shared/models/document.py](#).

Document state machine



- **DELETE** `/documents/{id}` is allowed in any state **except** **PROCESSING** (refusing while the worker holds it).
- **FAILED** is terminal — the worker writes an **error** message on the document; the frontend surfaces it.
- **EXTRACTED** is terminal in the current sprint; future tickets may re-open it for reprocessing.

Security model

- **API authentication.** Every authenticated endpoint requires **Authorization: Bearer** `<firebase_id_token>`. Tokens are verified server-side via Firebase Admin SDK; the decoded **uid** is the authorisation boundary.
- **Cross-user isolation.** Every read path takes **uid** as a parameter and returns **None** if a document belongs to another user. The repository layer enforces this, not the route handlers — the API can't accidentally bypass it. The HTTP response is **404**, not **403**, so existence can't be probed.
- **Direct GCS uploads via signed URLs.** Signed URLs are minted server-side; they encode the GCS object path, the exact **Content-Type**, a size cap (**x-goog-content-length-range**), and **if-generation-match: 0** (write-once). Any deviation by the client → GCS rejects with 403 or 412.
- **Worker authentication.** The `/pubsub/push` route requires a valid OIDC token from a dedicated push service account (**ailixir-pubsub-pusher**), verified for issuer, audience, **email**, and **email_verified**. The worker SA itself is separate from the pusher SA, so a compromise of one doesn't grant the other's privileges.
- **Least-privilege IAM.**
 - API SA: **storage.objectAdmin** on the documents bucket, **pubsub.publisher** on the topic, **datastore.user**, **firebaseauth.admin**, **iam.serviceAccountTokenCreator** on itself.
 - Worker SA: **storage.objectViewer** on the documents bucket (read-only), **storage.objectAdmin** on the cypher bucket (write), **datastore.user**, **aiplatform.user**, **documentai.apiUser**.
 - Pusher SA: only **run.invoker** on the worker service.
- **Secrets handling.** No secrets in the repo. CI/CD pulls Neo4j credentials and the Document AI processor ID from GitHub Secrets and passes them to Terraform. Firebase Admin and Vertex AI use Application Default Credentials in Cloud Run — no API keys on disk.

Reliability

- **Pub/Sub retry policy.** Failures retried with exponential backoff (10 s minimum, 600 s maximum). After 5 failed deliveries the message is routed to the `document-uploaded-dlq` dead-letter topic.
- **Atomic state transitions.** `mark_uploaded` uses a Firestore transaction with a status precondition so concurrent finalize attempts cannot both succeed.
- **Publish-after-commit.** The API only publishes the `DocumentUploaded` event *after* the Firestore commit. If the publish itself fails it is logged but does not roll back the document — the document remains in `uploaded` and a reconciliation job (future ticket) will re-publish.
- **Partial-upload tolerance.** If a multi-file document has some files missing in GCS at finalize time, only the present files get `upload_completed_at` set and `file_count` is updated to match. The pipeline still runs on what is present.
- **Soft delete with retention.** `DELETE /documents/{id}` writes a `deleted_at` timestamp; the bytes are not removed. A background reconciliation job (future) hard-deletes after the retention window. The endpoint refuses if the document is currently in `processing` so the worker isn't pulled out from under itself.

Areas Worth Improving

Documented so they can be tracked and fixed/improve in the future:

- **No worker-side idempotency.** Pub/Sub is at-least-once. Duplicate deliveries re-run the full pipeline (OCR cost paid twice, Graphiti episodes created twice). A `processed_events` collection keyed by `event_id` is the planned fix.
- **update_status has no precondition check.** Two parallel deliveries can both transition `uploaded` → `processing` and run the pipeline in parallel. Mitigated in practice by Pub/Sub's ordering on `document_id` but not eliminated.
- **DLQ has no subscriber.** Failed events accumulate in the dead-letter topic but nothing consumes them today. A subscriber that writes failures to an `extraction_failures` Firestore collection and marks the document `failed` is planned.
- **.env.example is partially stale.** It still references `OPENROUTER_API_KEY` and a Qwen vision model; the actual OCR backend is Google Cloud Document AI. The default `VERTEX_LLM_MODEL` listed (`gemini-2.0-flash-001`) differs from the deployed default (`gemini-2.5-flash-lite`). The deployed values are correct; the example file needs updating.
- **Neo4j and Document AI must be provisioned manually.** Terraform enables the APIs and the IAM roles, but doesn't create the Neo4j cluster (Aura or self-hosted) or the Document AI processor. Both are listed as one-time setup steps in the build documentation.
- **Cypher export silently truncates large graphs** (300-node, 500-edge cap in the Neo4j query). Acceptable for current document sizes; needs revisiting if individual documents start producing larger subgraphs.

Summary

The backend is built on standard managed Google Cloud building blocks — Cloud Run for compute, Cloud Storage for binary content, Firestore for metadata, Pub/Sub for async work, Document AI for OCR, and Vertex

AI Gemini (via Graphiti) for entity extraction. The split between a low-latency API that issues signed URLs and a worker that runs the heavy AI pipeline keeps the user-facing surface fast and the backend cost-bounded. Every piece is provisioned through Terraform and deployed through GitHub Actions, so the production environment can be rebuilt from `git clone` plus the four manual setup steps.